

NETWORK SECURITY

Final Research Report

# **Experimental Analysis and Secure Recovery of Cryptographic State Desynchronization in Network Communication**

Course: Network Security

Section: D

Prepared by

GHOSH, TUTUL — 23-50879-1

MD Tanvir Sajib — 22-46485-1

TARIKUL ISLAM — 20-43662-2

Department of Computer Science and Engineering  
American International University-Bangladesh

Prepared from the submitted research paper and project proposal.

# Executive Summary

This report presents a structured account of a research project on cryptographic state desynchronization in secure network communication. The central observation is that secure communication depends not only on strong cryptographic primitives, but also on consistent state between communicating endpoints. Session keys, nonce counters, sequence numbers, epochs, replay windows, and related metadata can become inconsistent when packets are lost, duplicated, delayed, reordered, or when a connection is interrupted or an endpoint restarts. The submitted research proposes a Lightweight State Consistency and Recovery (LSCR) layer that detects and classifies such divergence through authenticated state digests and uses fail-safe recovery decisions.

The proposed design is protocol-independent at the architectural level. It places an LSCR monitoring layer between the application and cryptographic engine, with four principal modules: state digest generation, divergence classification, recovery management, and session termination. The underlying experimental model uses UDP, AES-256-GCM for authenticated encryption, HKDF-SHA-256 for key derivation, and HMAC-SHA-256 for authenticated state verification.

The experiment defines twelve network scenarios using Linux `tc/netem` and programmatic fault injection. Each scenario uses 1,000 messages per trial, ten trials, and a message rate of 100 messages/second. The reported results show zero observed nonce reuse, replay acceptance, or invalid message acceptance across the reported experiment summaries, while the LSCR design introduces measurable normal-operation overhead. A critical qualification is that the tested scenarios did not actually trigger cryptographic state desynchronization, so the authenticated partial-recovery procedure itself was not exercised. This report therefore distinguishes validated observations from proposed capabilities and identifies concrete next steps for stronger recovery validation.

## 1. Introduction

Secure network communication requires communicating endpoints to maintain compatible cryptographic state throughout a session. The research identifies session identifiers, session keys, nonce counters, sequence numbers, epoch counters, replay windows, timestamps, and failure counters as state variables that influence message acceptance and continuity.

Network events can disturb this consistency without either endpoint being malicious. Packet loss can prevent state-advancing messages from arriving; duplication can cause repeated observations; reordering can make newer sequence numbers arrive first; delay can interact with timeout logic; and interruption or restart can break continuity assumptions. The research therefore treats cryptographic state consistency as a systems-security concern in addition to ordinary packet delivery.

The source literature review identifies a gap between protocol-specific mechanisms and a generalized mechanism for detecting, classifying, and safely recovering from network-induced cryptographic state divergence. LSCR is proposed as an additional architectural layer rather than a replacement for established protocols.

## 2. Research Objectives

- Develop a lightweight mechanism for detecting cryptographic state divergence without exposing sensitive cryptographic state.
- Classify divergence by severity and likely cause so bounded inconsistencies can be treated differently from suspicious or irrecoverable conditions.
- Use authenticated state digests to verify consistency without transmitting secret keys directly.
- Define fail-safe recovery: bounded divergence may trigger authenticated synchronization, while unsafe conditions trigger session termination and fresh key establishment.

- Measure security, reliability/availability, and performance against a conventional baseline.
- Evaluate behavior under packet loss, duplication, reordering, delay, interruption, restart, replay, and combined conditions.

### 3. Research Questions

- Can a lightweight, protocol-independent mechanism detect and recover from state divergence without weakening security guarantees?
- What performance overhead does the mechanism introduce in latency, CPU, memory, and communication?
- How does the mechanism compare with a baseline fail-closed/reconnect strategy under varying network conditions?

### 4. Background and Technical Foundation

Cryptographic state: The proposed endpoint state is  $S = (SID, SK, NC, SN_{send}, SN_{recv}, E, W, SD, T_{last}, F_{count})$ . These variables support identity binding, key use, freshness, ordering, replay protection, and state verification.

AES-256-GCM: used for authenticated encryption and confidentiality/integrity protection. HKDF-SHA-256: used for key derivation. HMAC-SHA-256: used for authenticated state-digest generation. Replay window: used to reject duplicate or stale sequence numbers while tolerating bounded reordering.

### 5. Literature Review

| Approach                 | Detection/state                 | Recovery                 | Limitation identified                         |
|--------------------------|---------------------------------|--------------------------|-----------------------------------------------|
| IPsec ESP                | Sequence number; sliding window | Discard                  | No general state recovery                     |
| RFID authentication      | Dual-state checks               | Old/new key              | Protocol-specific                             |
| Signal Double Ratchet    | Chain/DH state                  | Ratchet                  | Ordering complexity under loss/reordering     |
| WireGuard                | Counter; replay window          | Re-handshake             | No partial recovery                           |
| TLS 1.3                  | Key schedule/state transitions  | Reconnect                | TCP-based model                               |
| Noise Framework          | Handshake/authentication state  | Re-handshake             | No built-in partial recovery                  |
| AES-GCM-SIV              | Nonce misuse handling           | Deterministic encryption | Primitive-level, not recovery framework       |
| DTLS 1.3                 | Epoch/sequence; sliding window  | Retransmission/reconnect | Protocol-specific; no explicit classification |
| L-TESLA-CoAP             | Authentication key/pseudonym    | Edge synchronization     | Domain-specific; trusted edge                 |
| Automata-based analysis  | Protocol state                  | None                     | Detection-oriented                            |
| PUF-based authentication | PUF/key state                   | Hardware recovery        | Hardware dependency                           |
| Opossum attack           | Protocol state                  | None                     | Attack analysis, not recovery                 |

### 6. Research Gap

The reviewed mechanisms provide strong solutions for particular protocols, failure modes, or application domains, but the submitted research identifies a missing common layer combining explicit

state-consistency detection, multi-level divergence classification, and authenticated partial recovery. The proposed framework is intended to operate independently of a particular production protocol and to decide whether a state mismatch can be safely repaired or requires a fresh session.

This distinction matters academically: the work proposes an architectural abstraction that can later be adapted to TLS 1.3, DTLS 1.3, MQTT, CoAP, and other systems. Such integration and formal verification remain future work.

## 7. Baseline Framework

The baseline is a controlled client-server communication model over an unreliable UDP channel. Both endpoints maintain protocol state through a state manager, while a cryptographic engine performs encryption/decryption and key-related operations. A custom protocol is used to allow precise instrumentation and controlled experimentation.

The baseline security goals are confidentiality, integrity, authenticity, replay resistance, and secure recovery only when both endpoints are legitimate. The modeled network adversary can intercept traffic, drop packets, delay packets, reorder packets, duplicate packets, and observe packet sizes, timing, and metadata.

## 8. Proposed LSCR Architecture

The LSCR layer sits between the application and cryptographic engine. It contains four modules: State Digest Generator, Divergence Classifier, Recovery Manager, and Session Terminator. The endpoint state tuple contains SID, SK, NC, SNsend, SNrecv, E, W, SD, Tlast, and Fcount.

| Module                 | Responsibility                                                                                      |
|------------------------|-----------------------------------------------------------------------------------------------------|
| State Digest Generator | Computes an authenticated digest over relevant state without exposing the underlying secret state.  |
| Divergence Classifier  | Evaluates session ID, epoch, sequence-number, and nonce-counter differences and assigns a category. |
| Recovery Manager       | Performs authenticated state synchronization when the divergence is within configured safe bounds.  |
| Session Terminator     | Ends the session and triggers fresh key establishment when recovery cannot be trusted.              |

## 9. Divergence Classification

The classification procedure first checks session identity. A Session ID mismatch is irrecoverable. It then evaluates epoch drift, sequence-number drift, and nonce-counter drift against configured thresholds. The source design defines the following categories:

| Category          | Condition/cause                       | Action                                   |
|-------------------|---------------------------------------|------------------------------------------|
| Recoverable Minor | Loss/reordering within replay window  | Advance replay window and continue       |
| Recoverable Major | Significant but bounded gap           | Authenticated state synchronization      |
| Stale             | Severely delayed packet               | Discard and log                          |
| Suspicious        | Abnormal gap or possible attack       | Verify consistency; escalate if repeated |
| Irrecoverable     | Key/epoch mismatch beyond safe bounds | Terminate and establish new session      |
| Unknown           | Unclassifiable divergence             | Terminate and re-establish communication |

## 10. Secure Recovery Mechanism

For recoverable-major divergence, endpoint A generates a nonce challenge and sends a state synchronization request. Endpoint B verifies the request, computes sequence-number and nonce-counter gaps, and checks epoch equality. If all values are within configured recovery bounds, B returns an HMAC-authenticated nonce response and an acceptance message. A verifies the response, updates receiving sequence state, advances the replay window, verifies nonce consistency,

recomputes its state digest, and sends a synchronization confirmation.

If the recovery checks fail, the system rejects synchronization rather than forcing it. The session is terminated and a fresh key exchange is initiated. This fail-safe behavior is intended to prevent unsafe state progression.

## 11. Experimental Methodology

The research uses a controlled experimental client-server environment containing the client, server, cryptographic engine, state manager, UDP network layer, and network fault/attack injector. Linux tc/netem introduces reproducible loss, duplication, reordering, and delay, while interruption, endpoint restart, and replay are programmatically introduced.

Each scenario uses 1,000 messages per trial, ten trials, and a message rate of 100 messages/second. The evaluation considers security metrics (replay acceptance, nonce reuse, unsafe recovery), reliability metrics (delivery and session failure), and performance metrics (latency, CPU, memory, and communication overhead).

| ID  | Scenario            | Configuration                   |
|-----|---------------------|---------------------------------|
| S1  | Normal              | No impairment                   |
| S2  | 1% loss             | netem loss 1%                   |
| S3  | 5% loss             | netem loss 5%                   |
| S4  | 10% loss            | netem loss 10%                  |
| S5  | Duplication         | netem duplicate 5%              |
| S6  | Reordering          | 25%, gap 5                      |
| S7  | Delay               | 50 ms $\pm$ 20 ms               |
| S8  | Interruption        | 5-second programmatic link drop |
| S9  | Endpoint restart    | Programmatic state reset        |
| S10 | Replay              | External replay injection       |
| S11 | Loss + reorder      | 5% loss + 25% reorder, gap 5    |
| S12 | Loss + interruption | 5% loss + programmatic drop     |

## 12. Experimental Results

| Scenario             | Baseline        | LSCR            | Delta |
|----------------------|-----------------|-----------------|-------|
| S1 Normal            | 100.0%          | 100.0%          | 0.0%  |
| S2 1% loss           | 98.9 $\pm$ 0.2% | 99.1 $\pm$ 0.2% | +0.2% |
| S3 5% loss           | 95.2 $\pm$ 0.3% | 95.0 $\pm$ 0.6% | -0.1% |
| S4 10% loss          | 90.1 $\pm$ 0.6% | 90.6 $\pm$ 0.5% | +0.5% |
| S5 Duplication       | 100.0%          | 100.0%          | 0.0%  |
| S6 Reordering        | 100.0%          | 100.0%          | 0.0%  |
| S7 Delay             | 100.0%          | 98.0 $\pm$ 0.4% | -2.0% |
| S8 Interruption      | 100.0%          | 100.0%          | 0.0%  |
| S9 Restart           | 100.0%          | 100.0%          | 0.0%  |
| S10 Replay           | 100.0%          | 100.0%          | 0.0%  |
| S11 Loss + reorder   | 95.0 $\pm$ 0.4% | 95.1 $\pm$ 0.7% | +0.1% |
| S12 Loss + interrupt | 94.4 $\pm$ 0.4% | 95.1 $\pm$ 0.6% | +0.7% |

### 12.1 Security Findings

The reported experiments recorded zero security violations: zero nonce-reuse events, zero replay acceptances, and zero invalid-message acceptances in the reported experiment summaries. In S10, legitimate messages were delivered and replayed packets were rejected by the anti-replay mechanism.

### 12.2 Availability Findings

Most scenarios produced delivery close to the baseline. S7 was the main exception: LSCR recorded 98.0% delivery versus 100.0% for the baseline. The source attributes this to state-check timeout interactions with sustained delay. S12 recorded a 0.7 percentage-point LSCR improvement.

### 12.3 Performance Findings

| Metric                 | Baseline      | LSCR             | Change                 |
|------------------------|---------------|------------------|------------------------|
| Encryption latency     | 239.9 $\mu$ s | 267.5 $\mu$ s    | +27.6 $\mu$ s / +11.5% |
| Communication overhead | 72.0 B/msg    | 69.6 B/msg       | −3.3%                  |
| Client CPU             | 3.9%          | 5.1%             | +1.2 pp                |
| Client memory          | 29,088 KB     | 29,120 KB        | +32 KB / +0.1%         |
| Server CPU             | —             | Reported +0.6 pp | +0.6 pp                |
| Server memory          | —             | Reported +337 KB | +337 KB                |

## 13. Critical Interpretation

The most important qualification in the source paper is that no cryptographic state desynchronization was actually triggered by the tested scenarios. Packet loss mainly resulted in message non-delivery rather than confirmed cryptographic-state divergence. The paper explicitly states that the LSCR recovery protocol was therefore not implemented/exercised during the experiment. The experiments validate the architecture under common network conditions, but they do not directly establish recovery success for deliberately created cryptographic state divergence.

For an academic final submission, recovery effectiveness should therefore be described as a proposed and implemented mechanism whose direct recovery behavior still requires dedicated experimental validation.

## 14. Security Analysis

The security discussion uses a Dolev-Yao-style adversarial model and reports preservation of the baseline security properties for the tested cases. The design authenticates state information, checks session identity and epoch consistency, maintains nonce/sequence progression, and refuses recovery when the state gap exceeds safe boundaries.

The fail-safe branch is central: when state cannot be authenticated as safely recoverable, the session is terminated and fresh key establishment is initiated. The replay scenario further demonstrates rejection of replayed packets.

The source also acknowledges that the security argument relies on established cryptographic mechanisms and assumptions rather than a complete formal verification of LSCR. Hardware side channels, secure persistent state, large-scale concurrency, and heterogeneous deployment were not evaluated.

## 15. Strengths

- Clear separation of state-consistency management from the application and cryptographic engine.
- Authenticated digest approach avoids directly transmitting secret cryptographic state.
- Explicit multi-level divergence classification.

- Fail-safe distinction between bounded recovery and unsafe conditions.
- Joint security, availability, and performance evaluation.
- Reproducible network fault injection using tc/netem.

## 16. Limitations

- Controlled single-machine tc/netem testbed may not represent real-world network behavior.
- Point-to-point evaluation does not establish scalability across many concurrent sessions.
- No hardware evaluation for timing, power, or physical side channels.
- Actual cryptographic state divergence was not triggered, so partial recovery was not directly validated.
- Sustained delay caused state-check timeout sensitivity and session failures.
- Formal verification remains future work.

## 17. Future Work

- Integration with TLS 1.3 and DTLS 1.3.
- Adaptation for MQTT and CoAP.
- Evaluation on IoT and edge devices.
- Dedicated injection of recoverable, suspicious, stale, and irrecoverable state divergence.
- Formal verification using Tamarin/ProVerif.
- Large-scale Internet and heterogeneous-network testing.
- Secure persistent state storage and multi-party communication.
- Adaptive parameter tuning and comparison with QUIC.

## 18. Recommended Next Experimental Phase

The next phase should deliberately create cryptographic state divergence rather than only network impairment. One controlled method would be to advance one endpoint's sequence/nonce state while dropping the corresponding state-advancing communication, while keeping the session key and epoch valid. The experiment can then measure whether LSCR detects the divergence, classifies it correctly, authenticates recovery, restores synchronized state, and resumes communication without nonce reuse or replay acceptance.

| Test dimension | Measurement                                                         |
|----------------|---------------------------------------------------------------------|
| Detection      | True detection and false-positive rates                             |
| Classification | Correct category assignment                                         |
| Recovery       | Success rate and recovery latency                                   |
| Security       | Nonce reuse, replay acceptance, invalid acceptance, unsafe recovery |
| Availability   | Delivery and session failure before/after recovery                  |
| Performance    | CPU, memory, latency, synchronization-message cost                  |
| Stress         | Concurrent sessions, burst loss, longer delay, mixed faults         |

## 19. Conclusion

The research addresses a systems-security problem in which legitimate endpoints can lose cryptographic state consistency because of network events. The proposed LSCR architecture combines authenticated state digests, multi-level divergence classification, bounded synchronization, and fail-safe fresh-session establishment.

The reported evaluation shows relatively small resource overhead and preservation of the reported security invariants across the tested conditions. It also identifies timeout sensitivity under sustained delay. Most importantly, the experiment did not trigger actual cryptographic state divergence, so the recovery procedure needs direct experimental validation before recovery effectiveness can be treated as empirically established.

The work therefore provides a coherent research framework and a practical next step: deliberately generate controlled state divergence, measure recovery behavior, then extend the study through formal verification, concurrent-session testing, and deployment-oriented evaluation.

## 20. References

- [1] Diffie, W., & Hellman, M. (1976). New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6), 644–654.
- [2] Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446.
- [3] Kent, S. (2005). IP Encapsulating Security Payload (ESP). RFC 4303.
- [4] Kent, S., & Seo, K. (2005). Security Architecture for the Internet Protocol. RFC 4301.
- [5] Rescorla, E., Tschofenig, H., & Modadugu, N. (2022). The Datagram Transport Layer Security (DTLS) Protocol Version 1.3. RFC 9147.
- [6] Donenfeld, J. A. (2017). WireGuard: Next Generation Kernel Network Tunnel. NDSS.
- [7] Gope, P., & Hwang, T. (2016). A Realistic Lightweight Anonymous Authentication Protocol for Securing Real-Time Application Data Access in Wireless Sensor Networks. *IEEE Transactions on Industrial Electronics*.
- [8] Safkhani, M., & Bagheri, N. (2013). Generalized Desynchronization Attack on UMAP. *IEICE Transactions on Fundamentals*.
- [9] Dworkin, M. (2007). Recommendation for Block Cipher Modes of Operation: GCM and GMAC. NIST SP 800-38D.
- [10] Gueron, S., Langley, A., & Lindell, Y. (2019). AES-GCM-SIV: Nonce Misuse-Resistant Authenticated Encryption. RFC 8452.
- [11] van Deursen, T., & Radomirovic, S. (2009). Attacks on RFID Protocols. *Cryptology ePrint Archive*.
- [12] Sheffer, Y., Saint-Andre, P., & Fossati, T. (2022). Recommendations for Secure Use of TLS and DTLS. RFC 9325.
- [13] Perrin, T. (2018). The Noise Protocol Framework, Revision 34.
- [14] Perrin, T., & Marlinspike, M. (2016). The Double Ratchet Algorithm. Signal Foundation.
- [15] Avoine, G., Bingol, M. A., Carpent, X., & Yalcin, S. (2013). Privacy-Friendly Authentication in RFID Systems. *IEEE Transactions on Mobile Computing*.
- [16] Idriss, A., Bayoussef, H., & Asimi, A. (2024). A PUF-Based Lightweight Mutual Authentication Protocol for IoT Devices Using RFID Tags. *Electronics*.
- [17] Brinkmann, M., et al. (2025). The Opossum Attack: Exploiting Cross-Protocol Vulnerabilities in TLS. *USENIX Security Symposium*.
- [18] Fiterau-Brosteau, P., et al. (2023). Automata-Based Automated Detection of State Machine Bugs in Protocol Implementations. NDSS.
- [19] Firdous, A., Shahzad, S., & Kamran, M. (2024). Dynamic Session Enhanced Key Protocol for IoT Security. *arXiv preprint*.
- [20] Granjal, J., Monteiro, E., & Sa Silva, J. (2015). Security for the Internet of Things: A Survey of Existing Protocols and Open Research Issues. *IEEE Communications Surveys and Tutorials*.
- [21] Angrisani, K., Ferrara, A., & De Benedictis, M. (2023). L-TESLA-CoAP: Lightweight Authentication for Constrained IoT Devices. *Sensors*. The proposal source flags exact publication details for verification before final submission.